

☆

Singleton

| Guarantee exactly one instance of a class for the whole process — and one well-known way to reach it.

📍

DIVE 01 · 10 km

Airplane porthole

The big picture & contours

→

🏠

DIVE 02 · 1 km

Train window

Theses, examples, terms

→

♀

DIVE 03 · 100 m

Park walk

How the pieces relate

→

🔬

DIVE 04 · 1 cm

Microscope

Exact wording & gotchas

📍 Airplane porthole

DIVE 01 · ORIENTATION

▼ 10 km

The smallest creational pattern — it pins how many instances exist to exactly one, shared across the whole process.

— THE CONTOURS

• Collapses scattered globals
into one controlled access point

• Shared state, one identity
— every caller sees the same object

• Convenience now,
hidden coupling & test pain later

— USE IT WHEN — GoF applicability

🎯 Exactly one instance
must exist, reachable from one well-known access point

📦 Extensible by subclassing
and clients should use the chosen subclass without changing their code

📦 Creational family: Factory · Abstract Factory · Builder · Prototype · Singleton — Singleton answers “how many?”, not “which?”.

↓ DIVE DEEPER

🏠 Train window

DIVE 02 · KEY OBJECTS

▼ 1 km

— THE THREE PARTS

private constructor
blocks new from the outside — creation can't happen elsewhere

static instance
one cached field holds the sole object for the process

getInstance()
the single global door — lazily builds on first call, reuses after

— IN THE WILD

⚙️ java.lang.Runtime
getRuntime() — textbook JDK singleton

📦 Spring beans
singleton scope is the default

⌘ Node require / ESM
the module cache makes exports de-facto singletons

🖨️ Print spooler / driver
one shared device — GoF's own example

<> Client → Singleton.getInstance() → returns the cached static uniqueInstance.

| GoF intent: “Ensure a class has only one instance and provide a global point of access to it.”

↓ DIVE DEEPER

♀ Park walk

DIVE 03 · CONNECTIONS

▼ 100 m

Callers
A · B · C

→

getInstance()
single entry

→

instance == null ?
first call vs reuse

→

✓ one instance
returned to all

— WHY IT FOLLOWS

→ Private ctor ⇒ every creation path funnels through getInstance()

→ Static cache ⇒ first call builds it, all later calls reuse — identity preserved

→ Global access ⇒ no passing it around, but the dependency goes hidden

🔗 Abstract Factory / Builder / Facade are often Singletons · State & Strategy objects too · modern swap → a DI-scoped service.

— STRUCTURE · UML

Singleton

- instance: Singleton

- Singleton()

+ getInstance(): Singleton

+ operation(): void

getInstance() lazily creates, caches, then returns the one -instance; clients never call new.

— WHAT IT BUYS YOU

✔️ Controlled access — the class owns its instance; creation & access pass through one door

✔️ Reduced namespace — one access point instead of many global variables

✔️ Refinement by subclassing — choose the concrete instance at runtime, configured in one place

✔️ Variable instance count — the same shape generalizes to “exactly N”, not only one

✔️ Object, not statics — a real instance can be subclassed, mocked & passed — static methods can't

↓ DIVE DEEPER

🔬 Microscope

DIVE 04 · DETAILS

▼ 1 cm

| Definition: “Ensure a class has only one instance, and provide a global point of access to it.”
— GoF, “Design Patterns”, 1994

— THE CODE

```
class Config {
  static #instance?: Config;
  private constructor(readonly env = loadEnv()) {}
  static get(): Config {
    return (Config.#instance ??= new Config());
  }
}

// Node/ESM: a module's exports are cached per
// process → `export const cfg = new Config()`
// is already a singleton (no boilerplate).
```

— COMPARE THE ALTERNATIVES

Property	Singleton	Static	Module	DI scope
Lazy initialization	~	✗	✓	✓
One per process	✓	✓	✓	~
Mockable in tests	✗	✗	~	✓
Subclass / polymorphic	✓	✗	✗	✓
Explicit dependency	✗	✗	~	✓

Best fit → Singleton: legacy/OOP · Static: pure utilities · Module: Node/JS · DI: Nest/Spring. (“~” = partial / with caveats.)

— INTERVIEW PITFALLS

Q “Thread-safe by default?”
No — naive lazy getInstance() isn't. Use eager init, double-checked locking, the holder idiom, or an enum.

Q “Why an anti-pattern?”
It's hidden global state: hard to test, tight coupling, and an unclear lifecycle.

Q “How is it different from a static class?”
A Singleton is an object — it can implement interfaces, be subclassed and passed around; a static class can't.

Q “What about serverless?”
Instance state persists across warm invocations — it can leak or surprise between requests.

— VARIANTS

Eager
static instance = new X() — thread-safe by class-load, built even if unused

Holder idiom
init-on-demand holder class — lazy, lock-free (Java)

Module
Node/ESM exports cached per process — idiomatic JS

Lazy + lock
double-checked locking — lazy & thread-safe in JVM / CLR

Enum
serialization- & reflection-safe single instance (Effective Java)

DI scope
container owns one lifetime — testable; Nest default

— NUANCES & GOTCHAS

• Thread-safety — moot in single-thread Node, but worker_threads & duplicated module copies (dual-package hazard, multiple bundles) each mint their own → one-per-realm, not truly global

• Lazy vs eager — lazy pays on first touch & needs a guard (??=); eager is simpler but constructs unconditionally

• Testability trap — global mutable state leaks across tests, can't be reset, and hides dependencies → prefer DI

• It can be defeated — serialization & reflection can forge a second instance (readResolve / enum avoid it)

• Hard to extend — a static accessor isn't polymorphic, so subclassing & overriding get awkward

• Init-order bugs — hidden global access couples modules; circular first-touch can surprise you

— WATCH OUT

⚠️ Anti-pattern: Often an anti-pattern — a global in disguise. Default to a DI-scoped service or a plain module export; keep a hand-rolled Singleton only when a runtime/library truly demands one hard-guaranteed instance.

⚡ Modern take: ~90% of “singletons” should be a DI singleton-scoped service — same one-instance guarantee, but injectable, mockable and explicit about its dependencies.

● Vasyl Krupka · Senior Fullstack Engineer · learn-by-diving series

Source: GoF “Design Patterns” (1994)

v5